

For students

1a. 2 marks for each part with correct complexity and justification.

If direct correct answer is written without justification then 1.5 marks

If the answer is incorrect, then 0.5 mark for each loop correct complexity.

1b. Optimized algorithm 3 marks

Applying algorithm 6 Marks [1 mark for each iteration. In each iteration, focus on the stack contents, specifically the 4th column, and the current output array, which is the 6th column.]

Max 4 Marks given (2 Marks for algo + 2 Marks for execution using the given rain array) if the question is solved using another method which involves complexity more than $O(n)$.

2a

2b

3a Total - 6 marks (Handling edge cases (1M) – Empty list check + Correct head initialization (1M) – Proper dummy node or first node selection + Correct merging logic (3M) – Loop comparisons and pointer movement + Final attachment (1M) – Append remaining nodes if any)

If only two LLs are merged → 2 marks

If merged first and then sorted → 5 marks

3b Total - 9 marks (3 marks for Binary Search algo (recursive approach also considered) + 1 marks for insertion algo + 3 marks for BS apply + 1 marks insertion apply + 1 marks for Time Complexity)

4a Total - 5 marks (4Marks for applying selection sort + 0.5 mark for writing not stable + 0.5 mark for explanation or example).

If instead of applying, the correct algorithm is written then 1 mark is given out of 4 algorithm marks.

If other than selection algorithm is applied then 1 mark is given out of 4 algorithm marks.

4b 1 mark for each correct answer

QUESTION 1

Q1 a)

func1(int n) (2 marks) [1.5 marks if the correct answer is written without justification]

If the answer is wrong, 0.5 for the correct complexity corresponding to each loop

- Outer loop (i) runs from 1 to $n \rightarrow O(n)$.
- Middle loop (j) runs from 1 to $n \rightarrow O(n)$.
- Inner loop (k) runs from 1 to 10 $\rightarrow O(1)$ (constant).
- $T(n) = O(n) \times O(n) \times O(1) = O(n^2)$
- Time Complexity: $O(n^2)$

fun2(int n) (2 marks) [1.5 marks if the correct answer is written without justification]

If the answer is wrong, 0.5 for the correct complexity corresponding to each loop

- Outer loop (i) runs from 0 to $2n \rightarrow O(n)$.
- Middle loop (j) runs from 0 to $i/3$. In worst case, when $i \approx n$, it runs $O(n)$ times.
- Inner loop (k) runs from 0 to j^2 . Worst case when $j \approx n$, it runs $O(n^2)$ times.

So: $T(n) = O(n) \times O(n) \times O(n^2) = O(n^4)$, Time Complexity = $O(n^4)$

fun3(int n) (2 marks) [1.5 marks if the correct answer is written without justification]

If the answer is wrong, 0.5 for the correct complexity corresponding to each loop

$T(n) = O(n) \times O(n) \times O(n)$, Time Complexity = $O(n^3)$

Q1 b) (Total - 9 marks) [Max 4 Marks (2 Marks for algo + 2 Marks for execution using the given rain array) if the question is solved using another method which involves complexity more than $O(n)$]

Optimized Algorithm that returns the number of days one must wait until a future day with strictly greater rainfall occurs; (3 marks)

Algorithm NextGreaterRain(rain[0..n-1]):

Input: rain[0..n-1] // array of daily rainfall values
Output: out[0..n-1] // number of days until next greater rainfall

```
1. n ← length of rain
2. out[0..n-1] ← 0 // initialize output array
3. S ← empty stack // will store indices of days

4. for i ← n-1 down to 0 do
5.     while S is not empty AND rain[S.top()] ≤ rain[i] do
6.         S.pop()
7.     if S is not empty then
8.         out[i] ← S.top() - i
9.     else
10.        out[i] ← 0
```

```

11.     S.push(i)
12. return out

```

(6 Marks) [1 mark for each iteration. In each iteration, focus on the stack contents, specifically the 4th column, and the current output array, which is the 6th column.]

Iteration (i)	rain[i]	Action (pop/push)	Stack after processing	out[i] update	Current out[]
5	30	stack empty $\rightarrow$ no pop, push(5)	[5]	out[5] = 0	[0, 0, 0, 0, 0, 0]
4	9	rain[5]=30 > 9 $\rightarrow$ no pop, push(4)	[5, 4]	out[4] = 5-4=1	[0, 0, 0, 0, 1, 0]
3	18	rain[4]=9 $\leq$ 18 $\rightarrow$ pop(4), push(3)	[5, 3]	out[3] = 5-3=2	[0, 0, 0, 2, 1, 0]
2	25	rain[3]=18 $\leq$ 25 $\rightarrow$ pop(3), push(2)	[5, 2]	out[2] = 5-2=3	[0, 0, 3, 2, 1, 0]
1	15	rain[2]=25 > 15 $\rightarrow$ no pop, push(1)	[5, 2, 1]	out[1] = 2-1=1	[0, 1, 3, 2, 1, 0]
0	20	rain[1]=15 $\leq$ 20 $\rightarrow$ pop(1), push(0)	[5, 2, 0]	out[0] = 2-0=2	[2, 1, 3, 2, 1, 0]

Alternate Solution

Algorithm NextGreaterRain_Forward(rain[0..n-1]):
Input: rain[0..n-1]
Output: out[0..n-1]

```

1. n - length of rain
2. out[0..n-1] - 0
3. S - empty stack    // will store indices

4. for i - 0 to n-1 do
5.     while S is not empty AND rain[i] > rain[S.top()] do
6.         j - S.pop()
7.         out[j] - i - j
8.     S.push(i)
9. return out

```

Iteration (i)	rain[i]	Action (pop/push)	Stack after processing	out[j] update	Current out[]
0	20	stack empty $\rightarrow$ no pop, push(0)	[0]	-	[0, 0, 0, 0, 0, 0]
1	15	rain[1]=15 $\leq$ rain[0]=20 $\rightarrow$ no pop, push(1)	[0, 1]	-	[0, 0, 0, 0, 0, 0]
2	25	25 > rain[1]=15 $\rightarrow$ pop(1), out[1]=21=1; 25 > rain[0]=20 $\rightarrow$ pop(0), out[0]=20=2; push(2)	[2]	out[1]=1, out[0]=2	[2, 1, 0, 0, 0, 0]
3	18	18 $\leq$ rain[2]=25 $\rightarrow$ no pop, push(3)	[2, 3]	-	[2, 1, 0, 0, 0, 0]
4	9	9 $\leq$ rain[3]=18 $\rightarrow$ no pop, push(4)	[2, 3, 4]	-	[2, 1, 0, 0, 0, 0]
5	30	30 > rain[4]=9 $\rightarrow$ pop(4), out[4]=54=1; 30 > rain[3]=18 $\rightarrow$ pop(3), out[3]=53=2; 30 > rain[2]=25 $\rightarrow$ pop(2), out[2]=52=3; push(5)	[5]	out[4]=1, out[3]=2, out[2]=3	[2, 1, 3, 2, 1, 0]

QUESTION 2

Q2 a) (Total - 5 marks)

Token	Action	Stack	
60	push 60	[60]	
20	push 20	[60, 20]	
2	push 2	[60, 20, 2]	
/	pop 20,2 $\rightarrow 20 \div 2 = 10 \rightarrow$ push 10	[60, 10]	1 mark
+	pop 60,10 $\rightarrow 60 + 10 = 70 \rightarrow$ push 70	[70]	1 mark
15	push 15	[70, 15]	
5	push 5	[70, 15, 5]	
*	pop 15,5 $\rightarrow 15 \times 5 = 75 \rightarrow$ push 75	[70, 75]	1 mark
-	pop 70,75 $\rightarrow 70 - 75 = -5 \rightarrow$ push -5	[-5]	1 mark
100	push 100	[-5, 100]	
+	pop -5,100 $\rightarrow -5 + 100 = 95 \rightarrow$ push 95	[95]	1 mark

Give full marks for this type of solution also

2(a)

60	60
20	60, 20
2	60, 20, 2
/	60, (20/2)
+	(60 + (20/2))
15	(60 + (20/2)), 15
5	(60 + (20/2)), 15, 5
*	(60 + (20/2)), (15*5)
-	(60 + (20/2)) - (15*5)
100	(60 + (20/2)) - (15*5), 100
+	((60 + (20/2)) - (15*5)) + 100

$$((60 + (20/2)) - (15*5)) + 100$$

$$((60 + 10) - (75)) + 100$$

$$((70) - (75)) + 100$$

$$(-5 + 100)$$

$$= 95$$

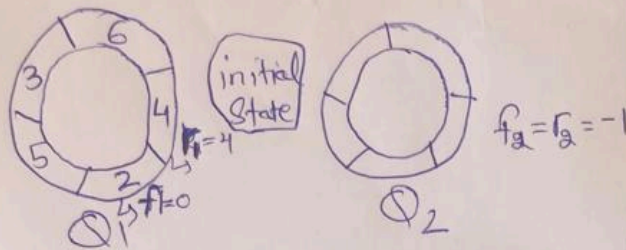
Q2 b) (Total - 10 marks)

(Total - 5 marks) Part 1: Push at rear, Pop from front	(Total - 5 marks) Part 2: Push at front, Pop from rear
--	--

Initial State: Q1: Contents = [2, 5, 3, 6, 4], front1 = 0, rear1 = 4 Q2: Empty front2 = -1, rear2 = -1 Iteration 1: (1 mark) Q1: Contents = [5, 3, 6, 4, 3], front1 = 1, rear1 = 0 Q2: Empty front2 = -1, rear2 = -1 Iteration 2: (1 mark) Q1: Contents = [3, 6, 4, 3], front1 = 2, rear1 = 0 Q2: Contents = [5], front2 = 0, rear2 = 0 Iteration 3: (1 mark) Q1: Contents = [6, 4, 3], front1 = 3, rear1 = 0 Q2: Contents = [5, 3], front2 = 0, rear2 = 1 Iteration 4: (1 mark) Q1: Contents = [4, 3, 6], front1 = 4, rear1 = 1 Q2: Contents = [5, 3], front2 = 0, rear2 = 1 Iteration 5: (1 mark) Q1: Contents = [3, 6, 5], front1 = 0, rear1 = 2 Q2: Contents = [5, 3], front2 = 0, rear2 = 1	Initial State: Q1: Contents = [2, 5, 3, 6, 4], front1 = 0, rear1 = 4 Q2: Empty front2 = -1, rear2 = -1 Iteration 1: (1 mark) Q1: Contents = [3, 2, 5, 3, 6], front1 = 4, rear1 = 3 Q2: Empty front2 = -1, rear2 = -1 Iteration 2: (1 mark) Q1: Contents = [3, 2, 5, 3], front1 = 4, rear1 = 2 Q2: Contents = [3], front2 = 0, rear2 = 0 Iteration 3: (1 mark) Q1: Contents = [3, 2, 5], front1 = 4, rear1 = 1 Q2: Contents = [3, 3], front2 = 4, rear2 = 0 Iteration 4: (1 mark) Q1: Contents = [3, 2], front1 = 4, rear1 = 0 Q2: Contents = [3, 3, 3], front2 = 3, rear2 = 0 Iteration 5: (1 mark) Q1: Contents = [3], front1 = 4, rear1 = 4 Q2: Contents = [3, 3, 3, 3], front2 = 2, rear2 = 0
--	--

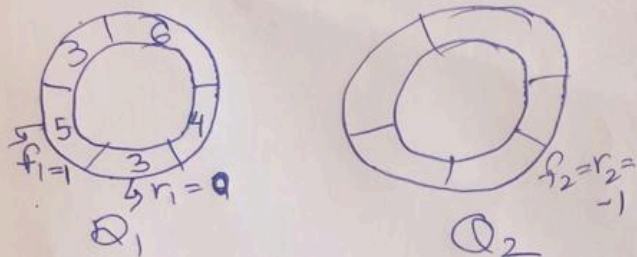
Part 1: Push at rear, Pop from front

Part 2: Push at front, Pop from rear

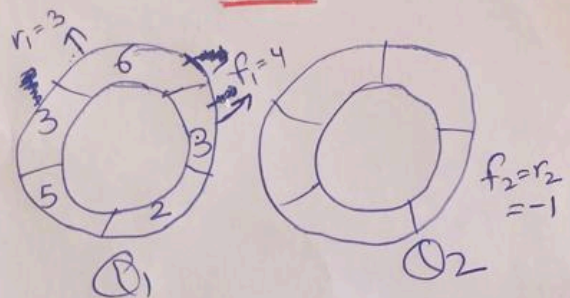


⇒ Same

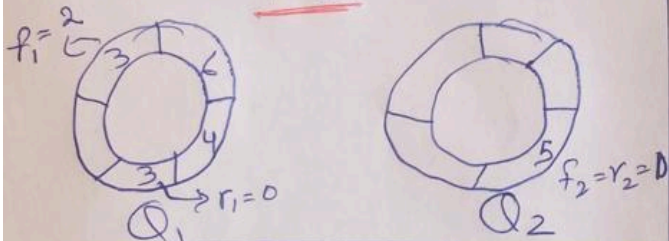
K=1



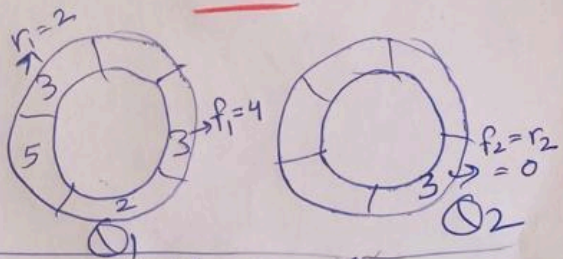
K=1



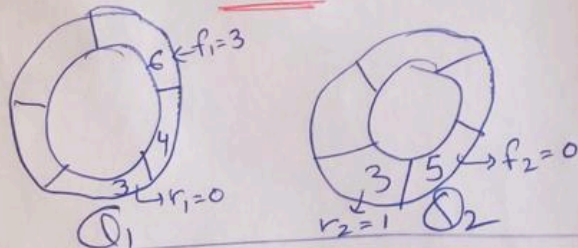
K=2



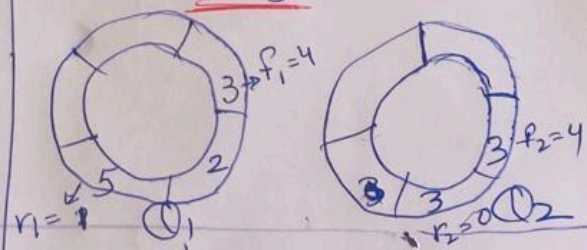
K=2



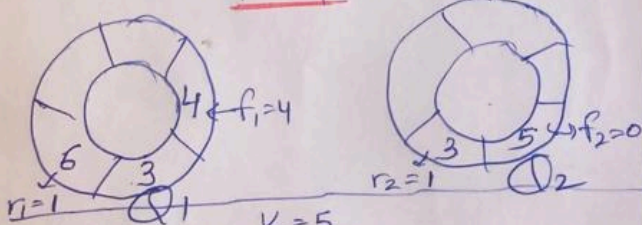
K=3



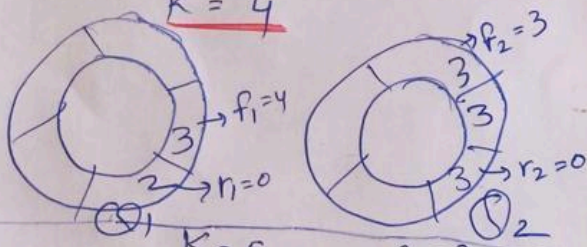
K=3



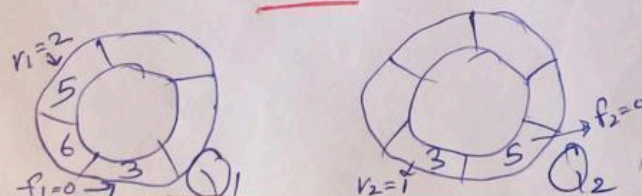
K=4



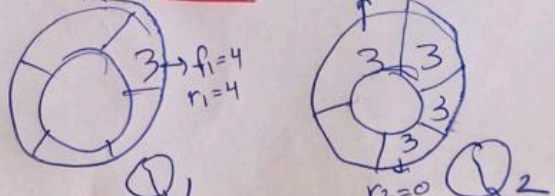
K=4



K=5



K=5



If every answer is wrong, then 1 mark out of 10 if a student tried to attempt and showed few iterations.

QUESTION 3

Q3 a) (Total - 6 marks)

Handling edge cases (1M) - Empty list handling: `if (!head1)` and `if (!head2)`

Correct initial head selection or dummy node (1M) - Proper initialization

Correct merging logic (3M) - Loop logic with comparisons and pointer movement

Final attachment of remaining nodes (1M) - If any list is left, append it to the merged list

Note:a) If simply 2 LLs are merged then 2 marks

b) If the two linked lists are merged first and then sorted, 5 marks are awarded.

```
Node* sortedMerge(Node* head1, Node* head2) {
    // Create a dummy node to simplify the merging process
    Node* dummy = new Node(-1);
    Node* curr = dummy;

    // Iterate through both linked lists
    while (head1 != nullptr && head2 != nullptr) {

        // Add the smaller node to the merged list
        if (head1->data <= head2->data) {
            curr->next = head1;
            head1 = head1->next;
        } else {
            curr->next = head2;
            head2 = head2->next;
        }
        curr = curr->next;
    }

    // If any list is left, append it to the merged list
    if (head1 != nullptr) {
        curr->next = head1;
    } else {
        curr->next = head2;
    }

    return dummy->next;
}
```



```
Algorithm MergeSortedLists(L1, L2)
Input: L1, L2 — heads of two sorted singly linked lists
Output: Head of the merged sorted singly linked list
```

```
if L1 == NULL return L2
```

```
if L2 == NULL return L1
```

```
# Initialize head of merged list
```

```
if L1.data <= L2.data then
```

```
    head ← L1
```

```
    L1 ← L1.next
```

```
else
```

```
    head ← L2
```

```
    L2 ← L2.next
```

```
current ← head
```

```
while L1 != NULL and L2 != NULL do
```

```
    if L1.data <= L2.data then
```

```
        current.next ← L1
```

```
        L1 ← L1.next
```

```
    else
```

```
        current.next ← L2
```

```
        L2 ← L2.next
```

```
    current ← current.next
```

```
# Attach the remaining nodes
```

```
if L1 != NULL then
```

```
    current.next ← L1
```

```
else
```

```
    current.next ← L2
```

```
return head
```

Q3 (b)

(Total - 9 marks)

3M for Binary Search algo (recursive approach also considered) + 1M for insertion algo + 3M for BS apply + 1M insertion apply + 1M for Time Complexity

Note: a) Linear search algo (1 marks) + Linear search apply (1 marks)

b) For the final answer, 1 mark;

c) For calling the array with values in the program, 1 mark.

d) Loop missing in Binary Search algo then 1 Marks will be deducted. (2 out of 3)

BINARY SEARCH TO FIND POSITION FOR INSERTION (3 marks)

findPosition(arr, n, x):

```
start = 0
end = n-1
pos = n // default insert at end
while start <= end:
    mid = (start + end) / 2
    if arr[mid] >= x:
        pos = mid
        end = mid - 1
    else:
        start = mid + 1
return pos
```

INSERTION ALGORITHM (1 mark)

insert(arr, n, x):

```
pos = findPosition(arr, n, x)
for i from n-1 downto pos:
    arr[i+1] = arr[i]
arr[pos] = x
return n+1
```

STEPS:

arr = {10,20,30,40,50} , n = 5 , x = 25

BINARY SEARCH (3 marks)

```
Start = 0, end = 4
mid=2 → arr[2]=30 ≥ 25 → pos = 2, end = 1
start=0, end=1
mid=0 → arr[0]=10 < 25 → start=1
mid=1 → arr[1]=20 < 25 → start=2
stops → pos=2
```

INSERTION (1 mark)

```
arr = {10, 20, 30, 40, 50, _}
shift from i=4→2
```

after shifting: {10, 20, 30, 30, 40, 50}
place 25 at pos=2
final = {10, 20, 25, 30, 40, 50}

TIME COMPLEXITY – $O(n)$ or $O(n + \log n)$ (1 mark)

QUESTION 4

Sol 4a (Total - 5 marks) - 4M for application + 0.5M for not stable + 0.5M for explanation or example

[If instead of apply, correct algorithm is written then 1M out of 4M is given.]

yield = [45, 36, 78, 10, 12]
Swap $a[0] \leftrightarrow a[3] \Rightarrow [10, 36, 78, 45, 12]$
Swap $a[1] \leftrightarrow a[4] \Rightarrow [10, 12, 78, 45, 36]$
Swap $a[2] \leftrightarrow a[4] \Rightarrow [10, 12, 36, 45, 78]$
Swap $a[3] \leftrightarrow a[3]$ (no change) $\Rightarrow [10, 12, 36, 45, 78]$
Sorted array: [10, 12, 36, 45, 78]

Max element placed at the end version:

yield = [45, 36, 78, 10, 12]
Swap $a[2] \leftrightarrow a[4] \Rightarrow [45, 36, 12, 10, 78]$
Swap $a[0] \leftrightarrow a[3] \Rightarrow [10, 36, 12, 45, 78]$
Swap $a[1] \leftrightarrow a[2] \Rightarrow [10, 12, 36, 45, 78]$
Swap $a[1] \leftrightarrow a[1]$ (no change)
Sorted array: [10, 12, 36, 45, 78]

No, the standard selection sort is not a stable sorting algorithm. It may change the relative order of equal keys because it performs a direct swap between the current position and the found minimum (which can move an earlier equal item behind a later equal item).

//If bubble/insertion sort is done in place of selection then 1 mark out of 4 assuming last iteration sorted output is correct.

Selection Algo and first iteration in detail written similarly other iterations so i considered as steps and 2 marks are given out of 4.

Sol 4b (Total - 10 marks)

1. curr->amount > or >= maxNode->amount
//if instead of amount, data/value written then also give 1 mark
// *curr->amount > *maxNode->amount Considered correct
 2. curr->amount < minNode->amount
 3. prev = curr; pre = prev-> next **is Wrong**
 4. curr = curr->next;
 5. curr != head curr->next != head is Wrong
 6. maxPrev->next = maxNode->next or //Correct maxPrev->next =maxPrev->next->next
 7. temp->next = maxNode;
 8. temp->next = newHead;
 9. minPre->next = minNode->next
minNode->next = minNode->next or //Correct minPrev->next =minPrev->next->next
 10. minNode->next = head or //Correct minNode->next = temp->next
-